

Startup Station CRM

Backend Architecture & Development Plan

v2.0 • Built from Live API Response Analysis • March 2026
Node.js • Express.js • MongoDB • Mongoose • Redis • BullMQ

1. Confirmed Technical Foundation

Every schema and API design in this document is derived directly from live API responses captured from `crm.startupstation.in`. No guesswork — field names, types, nesting, and relationships are all confirmed from real production data.

1.1 Tech Stack

Layer	Choice
Runtime	Node.js (LTS — v20 or v22)
Framework	Express.js
Database	MongoDB — confirmed via 24-char hex ObjectIDs in all URLs
ODM	Mongoose + mongoose-paginate-v2 plugin (matches live pagination format)
Auth	JWT — access token 15min + refresh token 7 days (httpOnly cookie)
Validation	Zod — every request body validated before reaching controller
Email	Nodemailer + SMTP/SendGrid — HTML templates with PDF attachments
File Storage	AWS S3 / Cloudflare R2 — PDFs, attachments
Job Queue	BullMQ + Redis — recurring invoice generation, followup reminders
Caching	Redis — dashboard aggregation cache (TTL 5 min)
API Versioning	/api/v1/ — matches live system exactly
Pagination	mongoose-paginate-v2 — matches live { docs, totalDocs, totalPages, page, hasPrevPage, hasNextPage, prevPage, nextPage }

1.2 Key Discoveries From Live API Analysis

Finding	Impact on Our Backend
state is ObjectId in list, full {name,id} object in detail	Always .populate('state') on detail endpoints, skip on list for performance
phone stored as Number not String	Mongoose: type: Number. Warning: numbers starting with 0 will lose the leading zero — match existing system behaviour
created/modified are Unix ms timestamps (not ISO dates)	Use type: Number for these fields to match. createdAt/updatedAt from Mongoose timestamps will be ISO — keep both
notes: [] is top-level alongside data, not inside it	Lead/customer detail response shape: { status, data: {...}, notes: [] }
customer is optional on Jobs	Some jobs are standalone with no customer linked — customerId is not required
compliance field optional on Jobs	Jobs auto-created from compliance have this ref; manually created jobs do not
Dashboard has leads + notes + sales + salesCount graphs	'notes' = daily interacted leads count — not a notes collection
FinancialYear is a separate collection	Not just a string — each year is a document with _id, name
ComplianceTemplate is a master catalog	Separate from CustomerCompliance — settings define the template, instances track per-customer status
Invoice has cgst/sgst/igst separately	Three separate tax fields + combined gst field on both invoice and invoice items
Invoice items are a separate collection	Not embedded in invoice — own collection with customer + invoice refs
linkedService + recurring refs on Invoice	Invoice links back to the customer service and recurring invoice that generated it
Email log is its own collection	Sent emails stored with full HTML content + template ref + subject
mongoose-paginate-v2 used for all lists	We must install and use this exact plugin to match the response format

2. Mongoose Schemas (100% From Live Data)

All schemas use { timestamps: true }. Soft deletes use deletedAt: { type: Date, default: null }. All list queries must include { deletedAt: null } in the filter.

2.1 Users

```
// Collection: users
const userSchema = new Schema({
  name:      { type: String, required: true },
  email:     { type: String, required: true, unique: true },
  password:  { type: String, required: true },    // bcrypt hashed
  role:      { type: String, enum: ['admin','agent','accountant'], default: 'agent' },
  active:    { type: Boolean, default: true },
  deletedAt: { type: Date, default: null },
}, { timestamps: true });

// Virtual for API response — return as { name, id }
userSchema.virtual('id').get(function() { return this._id; });
```

2.2 States

```
// Collection: states  (seeded once — all 28 states + 8 UTs of India)
// Confirmed: state is referenced by ObjectId, returned as { name, id } on detail
views
// Invoice state also includes 'code' field (GST state code)
const stateSchema = new Schema({
  name: { type: String, required: true },    // 'MAHARASHTRA', 'UTTAR PRADESH'
  code: { type: Number },                    // GST state code (confirmed from invoice
response)
});
```

2.3 Services

```
// Collection: services  (confirmed from live API)
const serviceSchema = new Schema({
  name:      { type: String, required: true },
  status:    { type: Boolean, default: true },
  template:  { type: Schema.Types.ObjectId, ref: 'EmailTemplate' },
  govt:      { type: Number, default: 0 },
  professional: { type: Number, default: 0 },
  hsn:       { type: String },
  deletedAt: { type: Date, default: null },
}, { timestamps: true });

// Denormalised snapshot stored on Lead/Customer: { name, id }
```

2.4 Leads

Confirmed fields from live list + detail responses. phone is Number (matches live system). state is ObjectId ref — populated as {name,id} on detail only.

```
// Collection: leads
const leadSchema = new Schema({
```

```

name:      { type: String, required: true },
phone:     { type: Number, required: true },    // Number - matches live system
companyName: { type: String, default: '' },      // Can be empty string
emails:    [{ type: String }],                  // Array, can be empty []
service: {
  name: { type: String },
  id:   { type: Schema.Types.ObjectId, ref: 'Service' }
},
source:    { type: String, enum:
['whatsapp','instagram','facebook','referral','website','others'] },
assigned: {                                     // Denormalised agent snapshot
  name: { type: String },
  id:   { type: Schema.Types.ObjectId, ref: 'User' }
},
type:      { type: String, enum: ['hot','cold'], default: 'cold' },
priority:  { type: String, enum: ['high','medium','low','none'] }, // Optional -
not always present
response:  { type: String, enum: ['interested','medium','not_interested','none']
}, // Optional
converted: { type: Boolean, default: false },
customerId: { type: Schema.Types.ObjectId, ref: 'Customer', default: null },
interacted: { type: Boolean, default: false },
address:   { type: String },
state:     { type: Schema.Types.ObjectId, ref: 'State' }, // Populated on detail
only
nextFollowup: { type: Date, default: null },
lastFollowup: { type: Number, default: null },    // Unix ms timestamp (matches
live)
created:     { type: Number },                    // Unix ms - set on create
modified:    { type: Number },                    // Unix ms - updated on every save
deletedAt:   { type: Date, default: null },
}, { timestamps: true });

// Indexes - all filterable fields
leadSchema.index({ 'assigned.id': 1 });
leadSchema.index({ type: 1, priority: 1 });
leadSchema.index({ source: 1 });
leadSchema.index({ converted: 1 });
leadSchema.index({ interacted: 1 });
leadSchema.index({ 'service.id': 1 });
leadSchema.index({ created: -1 });
leadSchema.index({ nextFollowup: 1 });
leadSchema.index({ lastFollowup: -1 });
leadSchema.index({ deletedAt: 1 });
leadSchema.index({ name: 'text', companyName: 'text', phone: 'text' });

leadSchema.plugin(mongoosePaginate);

```

2.5 Lead Notes (Followups)

```

// Collection: leadnotes
// Confirmed: notes: [] returned top-level on lead detail - separate collection
const leadNoteSchema = new Schema({
  leadId:    { type: Schema.Types.ObjectId, ref: 'Lead', required: true },
  content:   { type: String },
  date:      { type: Date },

```

```

    phoneCalled: { type: Boolean, default: false },
    createdBy:   { type: Schema.Types.ObjectId, ref: 'User' },
  }, { timestamps: true });

leadNoteSchema.index({ leadId: 1, createdAt: -1 });

```

2.6 Customers

Key naming differences from leads: 'saleBy' (not salesPerson), 'activeServices' (not services), 'compliance' boolean (not annualCompliance). FinancialYear is an array of refs to separate collection.

```

// Collection: customers
const customerSchema = new Schema({
  name:           { type: String, required: true },
  phone:          { type: Number, required: true },    // Number - matches live
  companyName:    { type: String, required: true },
  emails:         [{ type: String }],
  address:        { type: String },
  state:          { type: Schema.Types.ObjectId, ref: 'State' },
  gst:            { type: String, default: '' },
  type:           { type: String },    // 'Private Limited Company', 'LLP', 'OPC',
etc.
  incorporationDate: { type: Date },
  onboardingDate:    { type: Date },
  newlyIncorporated: { type: Boolean, default: false },
  username:          { type: String },
  password:          { type: String },    // Plain text - matches existing system
  saleBy: {
    name: { type: String },
    id:   { type: Schema.Types.ObjectId, ref: 'User' }
  },
  leadId:           { type: Schema.Types.ObjectId, ref: 'Lead', default: null },
  directors: [{
    // Embedded - bounded size
    name: { type: String, required: true },
    phone: { type: Number },
    email: { type: String },
    // _id auto-generated by Mongoose subdocument
  }],
  activeServices: [{
    // 'activeServices' - confirmed field name
    name: { type: String },
    id:   { type: Schema.Types.ObjectId, ref: 'Service' }
  }],
  compliance:      { type: Boolean, default: false },    // Annual compliance flag
  financialYear: [{
    // Array of refs to FinancialYear collection
    name: { type: String },    // '2025-2026'
    id:   { type: Schema.Types.ObjectId, ref: 'FinancialYear' }
  }],
  deletedAt:       { type: Date, default: null },
}, { timestamps: true });

customerSchema.index({ 'saleBy.id': 1 });
customerSchema.index({ compliance: 1 });
customerSchema.index({ createdAt: -1 });

```

```
customerSchema.index({ name: 'text', companyName: 'text', phone: 'text' });
customerSchema.plugin(mongoosePaginate);
```

2.7 Financial Years

```
// Collection: financialyears
// Confirmed: separate collection, referenced from customer.financialYear[]
// Also referenced from CustomerCompliance documents
const financialYearSchema = new Schema({
  name: { type: String, required: true, unique: true }, // '2025-2026', '2024-2025'
  startDate: { type: Date },
  endDate: { type: Date },
  active: { type: Boolean, default: true },
}, { timestamps: true });
```

2.8 Compliance Templates (Master Catalog)

```
// Collection: compliancetemplates (the 'Compliance Settings' section)
// Confirmed: compliance.id like '684970158660e5f4fc29771a' is the TEMPLATE
// These define WHAT needs to be done – not the per-customer status
const complianceTemplateSchema = new Schema({
  name: { type: String, required: true }, // 'Filing of Income Tax Returns'
  expiry: { type: Date }, // Fixed calendar expiry (ISO date)
  new: { type: Boolean, default: false }, // true = newly incorporated companies
  only
  days: { type: Number, default: null }, // Days from incorporation (for new:
  true)
  active: { type: Boolean, default: true },
}, { timestamps: true });

// When new=false: expiry is a fixed date (e.g. 2026-11-29)
// When new=true: expiry is calculated as incorporationDate + days
```

2.9 Customer Compliances

```
// Collection: customercompliances
// One document per customer per compliance template per financial year
// Confirmed structure from live compliance list response
const customerComplianceSchema = new Schema({
  customer: { // Denormalised snapshot
    name: { type: String },
    phone: { type: Number },
    companyName: { type: String },
    incorporationDate: { type: Date },
    id: { type: Schema.Types.ObjectId, ref: 'Customer' }
  },
  financialYear: { type: Schema.Types.ObjectId, ref: 'FinancialYear', required:
  true },
  compliance: { // Denormalised compliance template
    snapshot
    name: { type: String },
    expiry: { type: Date },
  }
```

```

    new:      { type: Boolean },
    days:     { type: Number },
    id:       { type: Schema.Types.ObjectId, ref: 'ComplianceTemplate' }
  },
  status:     { type: String, enum: ['To Be Done', 'Ongoing', 'Done', 'Expired'],
default: 'To Be Done' },
  accountant: { type: Schema.Types.ObjectId, ref: 'User', default: null },
  completedOn: { type: Date, default: null },
  deletedAt:  { type: Date, default: null },
}, { timestamps: true });

customerComplianceSchema.index({ 'customer.id': 1, financialYear: 1 });
customerComplianceSchema.index({ status: 1 });
customerComplianceSchema.index({ 'compliance.expiry': 1 });
customerComplianceSchema.plugin(mongoosePaginate);

```

2.10 Invoices

```

// Collection: invoices
// Confirmed: cgst/sgst/igst are separate fields. linkedService + recurring are refs.
// Items are a SEPARATE collection (not embedded).
// payments: [] is populated from Payments collection on detail view.
const invoiceSchema = new Schema({
  customer: { // Denormalised snapshot (full object on detail)
    name:      { type: String },
    phone:     { type: Number },
    companyName: { type: String },
    emails:    [{ type: String }],
    address:   { type: String },
    state:     { type: Schema.Types.ObjectId, ref: 'State' },
    gst:       { type: String },
    id:        { type: Schema.Types.ObjectId, ref: 'Customer' }
  },
  invoiceNo:   { type: String, required: true, unique: true }, // 'INV-24-2510938'
  price:       { type: Number, default: 0 }, // Subtotal before tax
  cgst:        { type: Number, default: 0 },
  sgst:        { type: Number, default: 0 },
  igst:        { type: Number, default: 0 },
  gst:         { type: Number, default: 0 }, // Total GST
  total:       { type: Number, default: 0 }, // Grand total
  due:         { type: Number, default: 0 }, // Amount still due
  invDate:     { type: Date },
  desc:        { type: String, default: '' },
  linkedService: { type: Schema.Types.ObjectId, ref: 'CustomerService', default: null },
}, {
  recurring:   { type: Schema.Types.ObjectId, ref: 'RecurringInvoice', default: null },
  emailTemplate: { // Snapshot of email template used
    content: { type: String },
    subject: { type: String },
    id:       { type: Schema.Types.ObjectId, ref: 'EmailTemplate' }
  },
  deletedAt:   { type: Date, default: null },
}, { timestamps: true });

invoiceSchema.index({ 'customer.id': 1 });

```

```
invoiceSchema.index({ invDate: -1 });
invoiceSchema.index({ due: 1 });
invoiceSchema.plugin(mongoosePaginate);
```

2.11 Invoice Items

```
// Collection: invoiceitems - separate collection, NOT embedded in invoice
// Confirmed from live invoice detail response
const invoiceItemSchema = new Schema({
  customer: { type: Schema.Types.ObjectId, ref: 'Customer', required: true },
  invoice: { type: Schema.Types.ObjectId, ref: 'Invoice', required: true },
  itemName: { type: String }, // 'Annual Compliance'
  product: {
    hsn: { type: String }, // '998399'
    id: { type: Schema.Types.ObjectId, ref: 'Service' }
  },
  professional: { type: Number, default: 0 },
  govt: { type: Number, default: 0 },
  price: { type: Number, default: 0 },
  cgst: { type: Number, default: 0 },
  cgstAmt: { type: Number, default: 0 },
  sgst: { type: Number, default: 0 },
  sgstAmt: { type: Number, default: 0 },
  igst: { type: Number, default: 0 },
  igstAmt: { type: Number, default: 0 },
  gst: { type: Number, default: 0 },
  total: { type: Number, default: 0 },
  created: { type: Number }, // Unix ms timestamp
}, { timestamps: true });

invoiceItemSchema.index({ invoice: 1 });
invoiceItemSchema.index({ customer: 1 });
```

2.12 Payments

```
// Collection: payments
// payments: [] returned on invoice detail - populated from this collection
const paymentSchema = new Schema({
  invoiceId: { type: Schema.Types.ObjectId, ref: 'Invoice', required: true },
  customerId: { type: Schema.Types.ObjectId, ref: 'Customer', required: true },
  amount: { type: Number, required: true },
  method: { type: String, enum: ['upi', 'card', 'bank_transfer', 'cash', 'other'] },
  note: { type: String },
  paidOn: { type: Date, default: Date.now },
  createdBy: { type: Schema.Types.ObjectId, ref: 'User' },
}, { timestamps: true });

paymentSchema.index({ invoiceId: 1 });
paymentSchema.index({ customerId: 1 });
paymentSchema.plugin(mongoosePaginate);
```


2.13 Recurring Invoices

```
// Collection: recurringinvoices
// Confirmed: 'recurring' field on Invoice refs this collection
const recurringInvoiceSchema = new Schema({
  customerId: { type: Schema.Types.ObjectId, ref: 'Customer', required: true },
  serviceId: { type: Schema.Types.ObjectId, ref: 'Service' },
  items:      [{ itemName: String, hsn: String, professional: Number, govt: Number,
price: Number, gst: Number, total: Number }],
  total:      { type: Number, default: 0 },
  frequency:  { type: String, enum: ['monthly','quarterly','yearly'] },
  nextDue:    { type: Date },
  active:     { type: Boolean, default: true },
  createdBy:  { type: Schema.Types.ObjectId, ref: 'User' },
}, { timestamps: true });

recurringInvoiceSchema.index({ nextDue: 1, active: 1 });
```

2.14 Accountant Jobs

Critical: customer is OPTIONAL — some jobs are standalone with no customer. compliance ref is also optional — only present when job was created from a compliance record.

```
// Collection: accountantjobs
// Confirmed from live /accountantjobs response
const jobSchema = new Schema({
  customer: { // OPTIONAL — some jobs have no customer
    name: { type: String },
    phone: { type: Number },
    companyName: { type: String },
    id: { type: Schema.Types.ObjectId, ref: 'Customer' }
  },
  jobTitle: { type: String, required: true }, // Free text — not enum
  accountant: { // Denormalised — always present
    name: { type: String },
    id: { type: Schema.Types.ObjectId, ref: 'User' }
  },
  status: { type: String, enum: ['To Be Done','Ongoing','Done','Expired'],
default: 'To Be Done' },
  expiryDate: { type: Date, default: null }, // Optional
  doneOn: { type: Date, default: null }, // Set when status -> Done
  compliance: { type: Schema.Types.ObjectId, ref: 'CustomerCompliance', default: null
}, // Optional
  created: { type: Date }, // ISO date string in live system
  deletedAt: { type: Date, default: null },
}, { timestamps: true });

jobSchema.index({ 'accountant.id': 1 });
jobSchema.index({ status: 1 });
jobSchema.index({ expiryDate: 1 });
jobSchema.index({ 'customer.id': 1 });
jobSchema.plugin(mongoosePaginate);
```

2.15 Email Templates

```
// Collection: emailtemplates
// Confirmed: used as ref in Service.template and Invoice.emailTemplate
// Subject uses {{placeholders}} – confirmed from live data
const emailTemplateSchema = new Schema({
  name: { type: String, required: true }, // 'Annual Compliance -
Onboarding'
  subject: { type: String, required: true }, // 'Welcome to Startup Station -
...'
  content: { type: String, required: true }, // Full HTML with {{placeholders}}
  attachmentKey: { type: String }, // S3 key for PDF attachment
  active: { type: Boolean, default: true },
}, { timestamps: true });

// Placeholders used: {{companyName}}, {{complianceName}}, {{complianceDoneDate}},
// {{name}}, {{address}}, {{invoiceNo}}, {{invoiceAmount}}, {{invoiceDate}}
// {{username}}, {{password}}, {{relationshipManager}}
```

2.16 Email Logs

```
// Collection: emaillogs
// Confirmed from live response – full HTML content stored per sent email
const emailLogSchema = new Schema({
  customer: { type: Schema.Types.ObjectId, ref: 'Customer' },
  lead: { type: Schema.Types.ObjectId, ref: 'Lead', default: null },
  type: { type: String, enum: ['customer','lead'], default: 'customer' },
  content: { type: String }, // Full rendered HTML that was sent
  template: {
    name: { type: String },
    id: { type: Schema.Types.ObjectId, ref: 'EmailTemplate' }
  },
  subject: { type: String },
  sentTo: [{ type: String }], // Emails it was sent to
  created: { type: Number }, // Unix ms timestamp
}, { timestamps: true });

emailLogSchema.index({ customer: 1, created: -1 });
emailLogSchema.index({ lead: 1, created: -1 });
```

2.17 Audit Log

```
// Collection: auditlogs – every create/update/delete recorded
const auditLogSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  userName: { type: String },
  entity: { type: String }, // 'Lead', 'Customer', 'Invoice' ...
  entityId: { type: Schema.Types.ObjectId },
  action: { type: String, enum: ['create','update','delete','convert','send_email']
},
  oldValue: { type: Schema.Types.Mixed },
  newValue: { type: Schema.Types.Mixed },
  ip: { type: String },
```

```
}, { timestamps: true });

auditLogSchema.index({ entity: 1, entityId: 1 });
auditLogSchema.index({ userId: 1, createdAt: -1 });
```

3. Complete API Route Map

All routes prefixed: <https://crm.startupstation.in/api/v1/>

3.1 Auth

Method + Endpoint	Description
POST /auth/login	{ email, password } → access token + refresh cookie
POST /auth/logout	Clears refresh cookie
POST /auth/refresh	Refresh cookie → new access token
GET /auth/me	Current user from token

3.2 Dashboard

Method + Endpoint	Description
GET /dashboard?from=&to=&agent=	Returns { data: { totalLeads, leadsNotInteracted, leadsInteracted, hotLeads, coldLeads, totalCustomers, complianceCustomer, totalInvoices, invTotal, invDue, countDues, totalPaymentCount, totalPaymentAmount, complianceToBeDone, complianceOngoing, expiredCompliance, jobToBeDone, expiredJobs, jobOngoing }, graph: { leads[], notes[], sales[], salesCount[] } }

3.3 Leads

Method + Endpoint	Description
GET /leads/list?page=&limit=&...	Paginated list — all filter params
POST /leads	Create lead
GET /leads/:id	Detail — { status, data: {...state populated}, notes: [] }
PUT /leads/:id	Update lead
DELETE /leads/:id	Soft delete

POST /leads/:id/convert	Convert to customer — atomic transaction
PUT /leads/:id/agent	Reassign agent
PUT /leads/:id/emails	Update emails array
POST /leads/:id/notes	Add note/followup
GET /leads/:id/notes	Get notes history
POST /leads/:id/send-email	Render + send email template
POST /leads/:id/send-whatsapp	WhatsApp template

3.4 Customers

Method + Endpoint	Description
GET /customers?page=&limit=&...	Paginated customer list
POST /customers	Create customer directly
GET /customers/:id	Full customer detail
PUT /customers/:id	Update customer
DELETE /customers/:id	Soft delete
POST /customers/:id/directors	Add director
PUT /customers/:id/directors/:did	Update director
DELETE /customers/:id/directors/:did	Remove director
POST /customers/:id/services	Add service
DELETE /customers/:id/services/:sid	Remove service
PUT /customers/:id/emails	Update emails
POST /customers/:id/financial-year	Add financial year → auto-create CustomerCompliance records
POST /customers/:id/send-email	Send email template
POST /customers/:id/send-whatsapp	Send WhatsApp template
GET /customers/:id/emails	Get sent email log
GET /customers/:id/director-report	Generate director report PDF

GET /customers/:id/board-resolution	Generate board resolution PDF
GET /customers/:id/consent-letter	Generate consent letter PDF
GET /customers/:id/auditors-report	Generate auditors report PDF

3.5 Compliances

Method + Endpoint	Description
GET /compliances?page=&limit=&financialYear=&customer=&status=	Paginated — matches live { docs, totalDocs, totalPages... }
PUT /compliances/:id	Update status, accountant, completedOn (the Modify button)
GET /compliance-templates	Master catalog list
POST /compliance-templates	Add new compliance type
PUT /compliance-templates/:id	Edit compliance type
DELETE /compliance-templates/:id	Remove from catalog
GET /financial-years	List all financial years
POST /financial-years	Add new financial year

3.6 Invoices

Method + Endpoint	Description
GET /invoices?page=&limit=&...	Paginated invoice list
POST /invoices	Create invoice + items in transaction
GET /invoices/:id	Detail — { data, items[], payments[], emailTemplate }
PUT /invoices/:id	Update invoice
DELETE /invoices/:id	Soft delete
POST /invoices/:id/send	Send invoice email
GET /invoices/number/next	Get next invoice number

3.7 Remaining Routes

Route Group	Key Endpoints
Payments	GET POST PUT DELETE /payments — CRUD + updates invoice.due on create
Recurring Invoices	GET POST PUT DELETE /recurring-invoices
Accountant Jobs	GET POST PUT DELETE /jobs — customer is optional on create
Email Templates	GET POST PUT DELETE /email-templates
Services	GET POST PUT DELETE /services
Users	GET POST PUT DELETE /users (admin only), GET /users/agents
States	GET /states (seeded, read-only)

4. Project Folder Structure

```
startup-station-backend/
├── src/
│   ├── config/
│   │   ├── db.js           # Mongoose connect
│   │   ├── redis.js        # Redis client + BullMQ setup
│   │   ├── s3.js           # AWS S3 / Cloudflare R2
│   │   └── mailer.js        # Nodemailer
│   ├── middleware/
│   │   ├── auth.js         # JWT verify
│   │   ├── checkRole.js    # Role-based guard
│   │   ├── validate.js     # Zod wrapper
│   │   ├── auditLog.js     # Auto-write to auditlogs after mutations
│   │   └── errorHandler.js # Global error handler
│   └── models/
│       ├── User.js
│       ├── State.js
│       ├── Service.js
│       ├── Lead.js
│       ├── LeadNote.js
│       ├── Customer.js
│       ├── FinancialYear.js
│       ├── ComplianceTemplate.js
│       ├── CustomerCompliance.js
│       ├── Invoice.js
│       ├── InvoiceItem.js
│       ├── Payment.js
│       ├── RecurringInvoice.js
│       ├── Job.js
│       ├── EmailTemplate.js
│       ├── EmailLog.js
│       └── AuditLog.js
```

```

├── modules/ # Feature modules
(routes/controller/service/queries)
├── auth/
├── dashboard/
├── leads/
├── customers/
├── compliances/
├── invoices/
├── payments/
├── recurring-invoices/
├── jobs/
├── email-templates/
├── services/
├── users/
├── workers/
├── recurringInvoice.worker.js
├── followupReminder.worker.js
├── utils/
├── buildLeadFilter.js # Dynamic query builder for lead filters
├── buildCustomerFilter.js
├── pdfGenerator.js # Director report, consent letter, board resolution
├── templateRenderer.js # Replace {{placeholders}} in email templates
├── invoiceNumber.js # Generate next invoice number INV-XX-XXXXXXX
├── denormalise.js # Helpers to build snapshot objects
├── app.js
├── server.js
├── .env
├── package.json

```

5. Critical Business Logic Workflows

5.1 Dashboard Aggregation

All 18 stats + 4 graph arrays come from a SINGLE endpoint. Must be cached in Redis with key: dashboard:{agentId}:{from}:{to}. TTL = 5 minutes. Agents only see their own data; admins see all.

```
GET /api/v1/dashboard?from=2026-03-01&to=2026-03-31&agent=
```

```
Run in parallel (Promise.all):
```

```

1) Lead.countDocuments({ created: {$gte: fromMs, $lte: toMs}, assigned.id: agentId })
2) Lead.countDocuments({ ...+ interacted: false })
3) Lead.countDocuments({ ...+ interacted: true })
4) Lead.countDocuments({ ...+ type: 'hot' })
5) Lead.countDocuments({ ...+ type: 'cold' })
6) Customer.countDocuments({ ... })
7) Customer.countDocuments({ ...+ compliance: true })

```

- 8) Invoice aggregate: count, sum(total), sum(due), count(due > 0)
- 9) Payment aggregate: count, sum(amount)
- 10) CustomerCompliance aggregate: count by status, count expired
- 11) Job aggregate: count by status, count expired

Graph arrays: group by date, count per day for leads/notes/sales/salesCount
→ MongoDB \$group by \$dateToString on createdAt
→ Fill missing dates with 0 to return full month array

5.2 Convert Lead → Customer (Atomic)

MUST use MongoDB session + transaction. If customer creation succeeds but lead update fails, you get orphaned data that is very hard to fix in production.

```
POST /api/v1/leads/:id/convert
body: { companyName, address, state, gst, type, incorporationDate,
       newlyIncorporated, username, onboardingDate }

const session = await mongoose.startSession();
session.startTransaction();
try {
  // 1. Validate lead exists + not already converted
  // 2. Generate password (random 8 char)
  // 3. Create Customer document (copy lead fields + new fields)
  // 4. Update Lead: converted=true, customerId=newCustomer._id
  // 5. Commit transaction
  // 6. Write AuditLog
  // 7. Return { status: true, data: { id: newCustomer._id } }
  //    → Frontend redirects to /customer/:id
} catch (err) {
  await session.abortTransaction();
  throw err;
}
```

5.3 Add Financial Year → Auto-Create Compliances

```
POST /api/v1/customers/:id/financial-year
body: { financialYearId }
```

1. Fetch customer (get incorporationDate, newlyIncorporated)
2. Fetch all active ComplianceTemplates
3. Filter templates:
 - If template.new === true: only include if newlyIncorporated === true
 - Calculate expiry: template.new ? incorporationDate + template.days : template.expiry
4. Bulk insert CustomerCompliance documents (one per template)
5. Push { name, id } to customer.financialYear[]
6. Return new compliance list

5.4 Invoice Number Generation

```
// Confirmed format from live data: 'INV-24-2510938'
// Pattern: INV-{YY}-{random 7 digits}
// Must be unique — check before assigning

async function generateInvoiceNumber() {
  const year = new Date().getFullYear().toString().slice(-2);
  let invoiceNo, exists;
  do {
    const rand = Math.floor(1000000 + Math.random() * 9000000);
    invoiceNo = `INV-${year}-${rand}`;
    exists = await Invoice.findOne({ invoiceNo });
  } while (exists);
  return invoiceNo;
}
```

5.5 Send Email Template

```
POST /api/v1/customers/:id/send-email
body: { templateId, customSubject?, customContent? }
```

1. Fetch customer (get emails[], name, companyName, ...)
2. Fetch EmailTemplate by templateId
3. Render: replace all {{placeholders}} with customer data
4. If template.attachmentKey → fetch PDF from S3
5. Send via Nodemailer to all customer.emails[]
6. Create EmailLog document (store full rendered HTML)
7. Write AuditLog
8. Return { status: true }

```
Placeholders: {{companyName}} {{name}} {{address}} {{username}}
               {{password}} {{complianceName}} {{complianceDoneDate}}
               {{invoiceNo}} {{invoiceAmount}} {{invoiceDate}}
```

5.6 Recurring Invoice Worker

```
// BullMQ CRON — runs daily at 06:00 AM

1. Find all: RecurringInvoice.find({ active: true, nextDue: { $lte: today } })
2. For each recurring invoice:
  a) generateInvoiceNumber()
  b) Create Invoice + InvoiceItem(s) in transaction
  c) Update RecurringInvoice.nextDue = next cycle date
3. Optional: send invoice email to customer

// nextDue calculation:
// monthly   → add 1 month
// quarterly → add 3 months
// yearly    → add 1 year
```

6. Security & Production Rules

6.1 Roles & Access

Role	What They Can Access	Restrictions
ADMIN	Everything — all agents' data, users, settings	None
AGENT	Own leads + customers only	Cannot see other agents' leads, cannot manage users
ACCOUNTANT	Compliances + Jobs + Invoices (read/write)	No access to leads, cannot manage users or settings

6.2 Non-Negotiable Rules

- Soft deletes on everything
- Audit log middleware
- Pagination always
- Zod on every request body
- Rate limiting on auth
- Standard error format
- Denormalisation on write
- MongoDB transactions
- Redis cache TTL

7. Development Phases

Build phases in order. Each phase produces fully working, tested endpoints before starting the next. Never skip a phase.

Phase	Deliverables
Phase 1 — Foundation (Week 1)	Mongoose connect, env config, error handler middleware, JWT auth (login/logout/refresh/me), Users CRUD, States seed + GET, Services CRUD
Phase 2 — Leads (Week 1-2)	Leads CRUD + all 9 filters + text search + pagination, Lead notes, Reassign agent, Update emails, Convert to customer (with transaction)

Phase 3 — Customers (Week 2)	Customers CRUD + filters, Directors CRUD (subdoc), ActiveServices add/remove, Emails update, Financial year add + auto-create compliances
Phase 4 — Compliance (Week 3)	ComplianceTemplate CRUD, FinancialYear CRUD, CustomerCompliance list + modify (status/accountant/completedOn), Compliance Settings page
Phase 5 — Billing (Week 3-4)	Invoice CRUD + number generation, InvoiceItem creation, Payments CRUD + invoice.due update, Recurring Invoices + BullMQ worker
Phase 6 — Jobs (Week 4)	Accountant Jobs CRUD (optional customer, optional compliance ref), job status updates
Phase 7 — Communication (Week 5)	Email Templates CRUD, Render + send email (Nodemailer), EmailLog storage, WhatsApp template send (WATI/Twilio API)
Phase 8 — Dashboard (Week 5)	All 18 aggregation stats, 4 graph arrays, Redis caching layer, date range + agent filter
Phase 9 — Documents (Week 6)	PDF generation (director report, consent letter, board resolution, auditors report), S3 upload/download
Phase 10 — Hardening (Week 6)	Full audit log review, rate limiting, input sanitisation, index review, load testing on 4200+ leads